

VERSÕES DO GESTOR IOT

versão 1

CODIGO C++

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "DHT.h"

// --- CONFIGURAÇÃO WI-FI ---
const char* ssid = "NOME_DO_SEU_WIFI";
const char* password = "SENHA_DO_WIFI";

WebServer server(80);
LiquidCrystal_I2C lcd(0x27, 16, 2);

#define PIN_R 4
#define PIN_G 17
#define PIN_B 16
#define RELAY_PIN 18
#define DHTPIN_INT 13
#define DHTPIN_EXT 12
#define DHTTYPE DHT22

DHT dhtInt(DHTPIN_INT, DHTTYPE);
DHT dhtExt(DHTPIN_EXT, DHTTYPE);

// --- PÁGINA WEB (DASHBOARD) ---
void handleRoot() {
    float tI = dhtInt.readTemperature();
    float tE = dhtExt.readTemperature();
    float hE = dhtExt.readHumidity();

    String corStatus = (tE < (tI - 2.0) && hE < 70.0) ? "#007bff" : (hE >= 70.0 ? "#ffc107" :
"#dc3545");
    String modoStr = (tE < (tI - 2.0) && hE < 70.0) ? "ECONOMIA ATIVA" : (hE >= 70.0 ?
"UMIDADE ALTA" : "MODO PADRAO");

    String html = "<html><head><meta name='viewport' content='width=device-width,
initial-scale=1' charset='utf-8'>";
    html += "<style>body{font-family:sans-serif; text-align:center; background:#f0f2f5;
margin:0;}";
    html += "header{background:#198754; color:white; padding:15px; font-size:20px;}";
    html += ".container{padding:20px;} .card{background:white; border-radius:15px;
padding:20px; margin:10px; box-shadow:0 4px 6px rgba(0,0,0,0.1); display:inline-block;
min-width:150px;}";
    html += ".status{padding:15px; border-radius:10px; color:white; font-weight:bold;
background:" + corStatus + "};</style>";
    html += "<script>setInterval(function(){location.reload();}, 3000);</script></head><body>";
    html += "<header>IFCE - Gestor Termico Bioclimatico</header>";
    html += "<div class='container'>";
```

```

    html += "<div class='card'><h3>Interno</h3><h2 style='color:#333;'>" + String(tI, 1) +
"C</h2></div>";
    html += "<div class='card'><h3>Externo</h3><h2 style='color:#333;'>" + String(tE, 1) +
"C</h2><p>Umidade: " + String(hE, 0) + "%</p></div>";
    html += "<div style='margin-top:20px;'><div class='status'>" + modoStr + "</div></div>";
    html += "</div></body></html>";

    server.send(200, "text/html", html);
}

void setup() {
    Serial.begin(115200);
    lcd.init();
    lcd.backlight();
    lcd.print("Conectando WiFi");

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nConectado! IP: ");
    Serial.println(WiFi.localIP());

    lcd.clear();
    lcd.print("IP: ");
    lcd.setCursor(0,1);
    lcd.print(WiFi.localIP()); // Mostra o IP no LCD para você digitar no celular
    delay(3000);

    server.on("/", handleRoot);
    server.begin();

    pinMode(PIN_R, OUTPUT);
    pinMode(PIN_G, OUTPUT);
    pinMode(PIN_B, OUTPUT);
    pinMode(RELAY_PIN, OUTPUT);
    dhtInt.begin();
    dhtExt.begin();
}

void loop() {
    server.handleClient();

    float tInt = dhtInt.readTemperature();
    float tExt = dhtExt.readTemperature();
    float hExt = dhtExt.readHumidity();

    if (isnan(tInt) || isnan(tExt)) return;

    // Monitoramento Local LCD
    lcd.setCursor(0, 0);
    lcd.print("I:"); lcd.print(tInt, 1); lcd.print("C E:"); lcd.print(tExt, 1); lcd.print("C");

```

```

lcd.setCursor(0, 1);
if (tExt < (tInt - 2.0)) {
  if (hExt < 70.0) {
    digitalWrite(RELAY_PIN, HIGH);
    definirCor(0, 0, 255); // Azul
    lcd.print("MOD0: ECONOMIA ");
  } else {
    digitalWrite(RELAY_PIN, LOW);
    definirCor(255, 255, 0); // Amarelo
    lcd.print("HUMID. ALTA ");
  }
} else {
  digitalWrite(RELAY_PIN, LOW);
  definirCor(255, 0, 0); // Vermelho
  lcd.print("MOD0: AR-COND. ");
}
}

void definirCor(int r, int g, int b) {
  analogWrite(PIN_R, r);
  analogWrite(PIN_G, g);
  analogWrite(PIN_B, b);
}

```

VERSÃO PARA APRESENTAR NA SALA (COM POTENCIÔMENTRO)

```

#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "DHT.h"

// --- CREDENCIAIS DO SEU WIFI REAL ---
const char* ssid = "NOME_DA_SUA_REDE";
const char* password = "SENHA_DA_REDE";

WebServer server(80);
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Definição de Pinos

```

```

#define PIN_R 4
#define PIN_G 17
#define PIN_B 16
#define RELAY_PIN 18
#define DHTPIN_INT 13
#define POT_EXT 34 // GPIO 34 é excelente para ADC
#define DHTTYPE DHT22

// Configuração PWM para o LED RGB (ESP32 real)
const int freq = 5000;
const int res = 8; // 8 bits (0-255)
const int canalR = 0;
const int canalG = 1;
const int canalB = 2;

DHT dhtInt(DHTPIN_INT, DHTTYPE);

float tI = 0, tE = 0, hE = 50.0;

void handleRoot() {
    String corStatus = "#dc3545";
    String modoStr = "MODO PADRAO (AC LIBERADO)";

    if (tE < (tI - 2.0)) {
        if (hE < 70.0) {
            corStatus = "#007bff";
            modoStr = "ECONOMIA ATIVA (VENTILACAO)";
        } else {
            corStatus = "#ffc107";
            modoStr = "SEGURANCA: UMIDADE ALTA";
        }
    }

    String html = "<html><head><meta name='viewport' content='width=device-width,
initial-scale=1' charset='utf-8'>";
    html += "<style>body{font-family:Arial; text-align:center; background:#f0f2f5; margin:0;};";
    html += "header{background:#198754; color:white; padding:20px; font-size:24px;
font-weight:bold;};";
    html += ".container{padding:20px;} .card{background:white; border-radius:15px;
padding:25px; margin:10px; box-shadow:0 4px 10px rgba(0,0,0,0.1); display:inline-block;
min-width:200px;};";
    html += ".status{padding:20px; border-radius:10px; color:white; font-size:1.2em;
font-weight:bold; margin-top:20px; background:" + corStatus + ";}</style>";
    html += "<script>setInterval(function(){location.reload();}, 3000);</script></head><body>";
    html += "<header>IFCE - GESTOR TERMICO IOT</header>";
    html += "<div class='container'>";
    html += "<div class='card'><h3>Temp. Interna</h3><h2 style='font-size:3em; color:#333;'>"
+ String(tI, 1) + "°C</h2></div>";
    html += "<div class='card'><h3>Temp. Externa</h3><h2 style='font-size:3em;
color:#333;'>" + String(tE, 1) + "°C</h2><p>Simulado via Potenciômetro</p></div>";
    html += "<div><div class='status'>" + modoStr + "</div></div>";
    html += "</div></body></html>";

    server.send(200, "text/html", html);
}

```

```

void setup() {
  Serial.begin(115200);

  // Configuração LED PWM
  ledcSetup(canalR, freq, res);
  ledcSetup(canalG, freq, res);
  ledcSetup(canalB, freq, res);
  ledcAttachPin(PIN_R, canalR);
  ledcAttachPin(PIN_G, canalG);
  ledcAttachPin(PIN_B, canalB);

  lcd.init();
  lcd.backlight();
  lcd.print("Conectando...");

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Conectado!");
  lcd.clear();
  lcd.print("IP: ");
  lcd.print(WiFi.localIP());

  server.on("/", handleRoot);
  server.begin();

  pinMode(RELAY_PIN, OUTPUT);
  pinMode(POT_EXT, INPUT);
  dhtInt.begin();
}

void loop() {
  server.handleClient();

  tl = dhtInt.readTemperature();

  // Leitura analógica com média simples para estabilidade
  int somaPot = 0;
  for(int i=0; i<10; i++) { somaPot += analogRead(POT_EXT); delay(5); }
  int leituraPot = somaPot / 10;

  // Mapeia 0-4095 para 15.0°C - 45.0°C
  tE = map(leituraPot, 0, 4095, 150, 450) / 10.0;

  if (isnan(tl)) return;

  lcd.setCursor(0, 0);
  lcd.print("Int:"); lcd.print(tl, 1); lcd.print(" Ext:"); lcd.print(tE, 1);

  lcd.setCursor(0, 1);
  if (tE < (tl - 2.0)) {

```

```

        digitalWrite(RELAY_PIN, HIGH);
        definirCor(0, 0, 255); // Azul
        lcd.print("ECONOMIA ATIVA ");
    } else {
        digitalWrite(RELAY_PIN, LOW);
        definirCor(255, 0, 0); // Vermelho
        lcd.print("MODO AR-CONDIC. ");
    }
    delay(1000);
}

void definirCor(int r, int g, int b) {
    ledcWrite(canalR, r);
    ledcWrite(canalG, g);
    ledcWrite(canalB, b);
}

```

VERSÃO 2 COM POTENCIOMETRO

```

#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "DHT.h"

// =====
// 1. CONFIGURAÇÕES E PINOS
// =====

const char* wifi_privado = "NOME_DA_REDE";
const char* pass_privado = "SENHA_DA_REDE";
const char* wifi_aberto = "REDE_ABERTA_IFCE";

// Configuração do modo "Sem Internet" (ESP32 cria o Wi-Fi)
const char* ap_nome = "PROJETO_EFICIENCIA";
const char* ap_pass = "12345678";

// Pinos de Conexão
#define PIN_RED 4
#define PIN_GREEN 2
#define PIN_BLUE 15
#define PIN_RELAY 5
#define PIN_DHT 33
#define PIN_POT 34 // Potenciômetro (Temperatura Externa)
#define DHTTYPE DHT22

```

```

// Objetos
WebServer server(80);
LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht_interno(PIN_DHT, DHTTYPE);

// Variáveis de leitura
float temp_interna = 0;
float temp_externa = 0;

// =====
// 2. DASHBOARD WEB (HTML/CSS)
// =====
void pagina_dashboard() {
    // Lógica de decisão para o site
    bool economia = (temp_externa < (temp_interna - 2.0));
    String cor_box = economia ? "#007bff" : "#dc3545"; // Azul ou Vermelho
    String texto_status = economia ? "ECONOMIA ATIVA: Ventilação
Externa" : "MODO PADRÃO: Ar-Condicionado";

    String html = "<!DOCTYPE html><html lang='pt-br'><head><meta
charset='utf-8'>";
    html += "<meta name='viewport' content='width=device-width,
initial-scale=1.0'>";
    html += "<style>body{font-family:Arial; text-align:center;
background:#eee;}";
    html += ".card{background:white; margin:10px; padding:20px;
border-radius:15px; box-shadow:0 4px 8px rgba(0,0,0,0.1);
display:inline-block; width:250px;}";
    html += ".status{padding:20px; color:white; font-weight:bold;
background:" + cor_box + "};</style>";
    html += "<script>setInterval(() => { location.reload(); }, 3000);</script>";
// Auto-refresh 3s
    html += "<title>Gestão Térmica IoT</title></head><body>";

    html += "<h1>IFCE - Eficiência Energética</h1>";
    html += "<div class='card'><h3>Temp. Interna</h3><h2>" +
String(temp_interna, 1) + "°C</h2></div>";
    html += "<div class='card'><h3>Temp. Externa</h3><h2>" +
String(temp_externa, 1) + "°C</h2></div>";
    html += "<div class='status'><h3>" + texto_status + "</h3></div>";

    html += "</body></html>";
    server.send(200, "text/html", html);

```

```

}

// =====
// 3. FUNÇÕES DE SUPORTE
// =====

void gerenciar_led_rgb(int r, int g, int b) {
    // ESP32 usa PWM para controlar a intensidade das cores
    ledcWrite(0, r);
    ledcWrite(1, g);
    ledcWrite(2, b);
}

void conectar_wifi() {
    lcd.clear();
    lcd.print("Buscando WiFi...");

    // Tenta rede Privada
    WiFi.begin(wifi_privado, pass_privado);
    int tentativas = 0;
    while (WiFi.status() != WL_CONNECTED && tentativas < 10) {
        delay(500);
        tentativas++;
    }

    // Se falhar, tenta rede Aberta
    if (WiFi.status() != WL_CONNECTED) {
        WiFi.begin(wifi_aberto, "");
        tentativas = 0;
        while (WiFi.status() != WL_CONNECTED && tentativas < 10) {
            delay(500);
            tentativas++;
        }
    }

    // Se tudo falhar, cria o próprio WiFi (Access Point)
    if (WiFi.status() != WL_CONNECTED) {
        WiFi.softAP(ap_nome, ap_pass);
        lcd.clear();
        lcd.print("WiFi Proprio:");
        lcd.setCursor(0,1);
        lcd.print(WiFi.softAPIP());
    } else {
        lcd.clear();
    }
}

```



```

    lcd.print("Conectado IP:");
    lcd.setCursor(0,1);
    lcd.print(WiFi.localIP());
}
}

// =====
// 4. SETUP E LOOP PRINCIPAL
// =====
void setup() {
    Serial.begin(115200);

    // Configura os canais PWM para o LED RGB
    ledcSetup(0, 5000, 8); ledcSetup(1, 5000, 8); ledcSetup(2, 5000, 8);
    ledcAttachPin(PIN_RED, 0); ledcAttachPin(PIN_GREEN, 1);
    ledcAttachPin(PIN_BLUE, 2);

    pinMode(PIN_RELAY, OUTPUT);
    pinMode(PIN_POT, INPUT);

    lcd.init();
    lcd.backlight();
    dht_interno.begin();

    conectar_wifi();

    server.on("/", pagina_dashboard);
    server.begin();
}

void loop() {
    server.handleClient(); // Mantém o site no ar

    // A. LEITURA DOS SENSORES
    temp_interna = dht_interno.readTemperature();

    // B. SIMULAÇÃO DA TEMP EXTERNA VIA POTENCIÔMETRO
    // Converte valor 0-4095 para 15.0-45.0 graus Celsius
    int leitura_pot = analogRead(PIN_POT);
    temp_externa = map(leitura_pot, 0, 4095, 150, 450) / 10.0;

    if (isnan(temp_interna)) return; // Evita erro de leitura

```

```

// C. LÓGICA DE EFICIÊNCIA ENERGÉTICA (O CORAÇÃO DO
PROJETO)
// Se a externa for pelo menos 2 graus menor que a interna
if (temp_externa <= (temp_interna - 2.0)) {
    // MODO ECONOMIA: Desliga AC (Relé) e sinaliza AZUL
    digitalWrite(PIN_RELAY, LOW);
    gerenciar_led_rgb(0, 0, 255); // Azul

    lcd.setCursor(0,0); lcd.print("I:"); lcd.print(temp_interna,1); lcd.print("
E:"); lcd.print(temp_externa,1);
    lcd.setCursor(0,1); lcd.print("MODO: ECONOMIA ");
}
else {
    // MODO PADRÃO: Liga AC (Relé) e sinaliza VERMELHO
    digitalWrite(PIN_RELAY, HIGH);
    gerenciar_led_rgb(255, 0, 0); // Vermelho

    lcd.setCursor(0,0); lcd.print("I:"); lcd.print(temp_interna,1); lcd.print("
E:"); lcd.print(temp_externa,1);
    lcd.setCursor(0,1); lcd.print("MODO: AR-CONDIC.");
}

delay(500); // Estabilidade
}

```

VERSÃO 3 COM POTENCIOMETRO E MOTOR DC

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "DHT.h"

// =====
// 1. CONFIGURAÇÕES E PINOS
// =====
const char* wifi_privado = "Rafael";
const char* pass_privado = "10021972";
const char* wifi_aberto = "IFCE-PECEM";

// Configuração do modo "Sem Internet" (ESP32 cria o Wi-Fi)
const char* ap_nome = "PROJETO_EFICIENCIA";
const char* ap_pass = "12345678";

// Pinos de Conexão
#define PIN_RED 4
#define PIN_GREEN 2
#define PIN_BLUE 15
#define RELE_AC 5
#define PIN_DHT 23 // Mantido conforme sua última atualização
#define PIN_POT 34 // Potenciômetro (Temperatura Externa)
#define DHTTYPE DHT22
#define RELE_MOTOR 33 // Mantido conforme sua última
atualização

// Objetos
WebServer server(80);
LiquidCrystal_I2C lcd(0x27, 16, 2); // Caso não funcione, teste
mudar 0x27 para 0x3F
DHT dht_interno(PIN_DHT, DHTTYPE);

// Variáveis de leitura
float temp_interna = 0;
float temp_externa = 0;
String ip_atual = ""; // Armazena o IP para exibir no LCD
```

```
// =====
// 2. DASHBOARD WEB (HTML/CSS)
// =====

void pagina_dashboard() {
    bool economy = (temp_externa < (temp_interna - 2.0));
    String cor_box = economy ? "#007bff" : "#dc3545"; // Azul ou
    Vermelho
    String texto_status = economy ? "ECONOMIA ATIVA: Ventilação
    Externa" : "MODO PADRÃO: Ar-Condicionado";

    String html = "<!DOCTYPE html><html lang='pt-br'><head><meta
    charset='utf-8'>";
    html += "<meta name='viewport' content='width=device-width,
    initial-scale=1.0'>";
    html += "<style>body{font-family:Arial; text-align:center;
    background:#eee;}";
    html += ".card{background:white; margin:10px; padding:20px;
    border-radius:15px; box-shadow:0 4px 8px rgba(0,0,0,0.1);
    display:inline-block; width:250px;}";
    html += ".status{padding:20px; color:white; font-weight:bold;
    background:" + cor_box + "};</style>";
    html += "<script>setInterval(() => { location.reload(); },
    3000);</script>"; // Auto-refresh 3s
    html += "<title>Gestão Térmica IoT</title></head><body>";

    html += "<h1>IFCE - Eficiência Energética</h1>";
    html += "<div class='card'><h3>Temp. Interna</h3><h2>" +
    String(temp_interna, 1) + "°C</h2></div>";
    html += "<div class='card'><h3>Temp. Externa</h3><h2>" +
    String(temp_externa, 1) + "°C</h2></div>";
    html += "<div class='status'><h3>" + texto_status + "</h3></div>";

    html += "</body></html>";
    server.send(200, "text/html", html);
}

// =====
// 3. FUNÇÕES DE SUPORTE
// =====
```

```

void gerenciar_led_rgb(int r, int g, int b) {
  analogWrite(PIN_RED, r);
  analogWrite(PIN_GREEN, g);
  analogWrite(PIN_BLUE, b);
}

void conectar_wifi() {
  lcd.clear();
  lcd.print("1.Tentando Priv.");

  // Tenta rede Privada
  WiFi.begin(wifi_privado, pass_privado);
  int tentativas = 0;
  while (WiFi.status() != WL_CONNECTED && tentativas < 10) {
    delay(500);
    tentativas++;
  }

  // Se falhar, tenta rede Aberta
  if (WiFi.status() != WL_CONNECTED) {
    lcd.clear();
    lcd.print("2.Tentando Aber.");
    WiFi.begin(wifi_aberto, "");
    int tentativas_aberto = 0;
    while (WiFi.status() != WL_CONNECTED && tentativas_aberto <
10) {
      delay(500);
      tentativas_aberto++;
    }
  }

  // Se tudo falhar, cria o próprio WiFi (Access Point)
  if (WiFi.status() != WL_CONNECTED) {
    WiFi.softAP(ap_nome, ap_pass);
    ip_atual = WiFi.softAPIP().toString();
    lcd.clear();
    lcd.print("3.Modos AP Ativo");
  } else {
    ip_atual = WiFi.localIP().toString();
    lcd.clear();
  }
}

```

```

    lcd.print("WiFi Conectado!");
}
delay(1500); // Dá tempo para ler a mensagem de conexão na tela
}

// =====
// 4. SETUP E LOOP PRINCIPAL
// =====
void setup() {
    Serial.begin(115200);

    pinMode(PIN_RED, OUTPUT);
    pinMode(PIN_GREEN, OUTPUT);
    pinMode(PIN_BLUE, OUTPUT);

    pinMode(RELE_AC, OUTPUT);
    pinMode(RELE_MOTOR, OUTPUT);
    pinMode(PIN_POT, INPUT);

    pinMode(PIN_DHT, INPUT); // Configuração para resistor físico
    externo

    lcd.init();
    lcd.backlight();
    dht_interno.begin();

    conectar_wifi();

    server.on("/", pagina_dashboard);
    server.begin();
}

void loop() {
    server.handleClient(); // Mantém o site no ar

    // A. LEITURA DOS SENSORES
    temp_interna = dht_interno.readTemperature();

    // B. SIMULAÇÃO DA TEMP EXTERNA VIA POTENCIÔMETRO
    int leitura_pot = analogRead(PIN_POT);

```

```

temp_externa = map(leitura_pot, 0, 4095, 150, 450) / 10.0;

// --- C. LÓGICA DE EFICIÊNCIA ENERGÉTICA ---
if (temp_externa <= (temp_interna - 2.0)) {
    // MODO ECONOMIA: Desliga AC, Liga Ventilação Mecânica e
    sinaliza AZUL
    digitalWrite(RELE_AC, HIGH);    // DESLIGA o Ar-Condicionado
    (Relé 1)
    digitalWrite(RELE_MOTOR, LOW); // LIGA O MOTOR /
    VENTILADOR (Relé 2)
    gerenciar_led_rgb(0, 0, 255);    // Liga a cor Azul pura
}
else {
    // MODO PADRÃO: Liga AC, Desliga Ventilação Mecânica e
    sinaliza VERMELHO
    digitalWrite(RELE_AC, LOW);      // LIGA o Ar-Condicionado
    (Relé 1)
    digitalWrite(RELE_MOTOR, HIGH); // DESLIGA O MOTOR /
    VENTILADOR (Relé 2)
    gerenciar_led_rgb(255, 0, 0);    // Liga a cor Vermelha pura
}

// --- D. ATUALIZAÇÃO DO DISPLAY LCD (TRAVADO SEM
ALTERNÂNCIA) ---
// Linha 1 fixa: IP do servidor
lcd.setCursor(0,0);
lcd.print("IP:" + ip_atual);

// Preenche com espaços vazios até o fim da linha para limpar
resíduos antigos
for (int i = ("IP:" + ip_atual).length(); i < 16; i++) {
    lcd.print(" ");
}

// Linha 2 fixa: Temperaturas Interna e Externa
lcd.setCursor(0,1);
lcd.print("I:"); lcd.print(temp_interna, 1); lcd.print("C E:");
lcd.print(temp_externa, 1); lcd.print("C ");

delay(200);

```

```
}
```

versão 4

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "DHT.h"

// --- CREDENCIAIS DAS REDES ---
const char* wifi_privado = "Navegamais";
const char* pass_privado = "TonyS@les20";
const char* wifi_aberto = "IFCE-PECEM";

// Configuração do modo "Sem Internet" (Access Point)
const char* ap_nome = "PROJETO_EFICIENCIA";
const char* ap_pass = "12345678";

WebServer server(80);
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Definição de Pinos
#define PIN_R 4
#define PIN_G 2
#define PIN_B 15
#define RELE_AC 5 // Seu Relé do Ar-Condicionado
#define RELE_MOTOR 33 // Seu Relé do Motor / Ventilador
#define DHTPIN_INT 23
#define POT_EXT 34 // GPIO 34 para o Potenciômetro
#define DHTTYPE DHT22

DHT dhtInt(DHTPIN_INT, DHTTYPE);

float tI = 25.0, tE = 0, hE = 50.0;
String ip_atual = "";

// =====
// DASHBOARD WEB (HTML/CSS)
```



```
// =====
void handleRoot() {
    String corStatus = "#dc3545";
    String modoStr = "MODO PADRAO (AC LIBERADO)";

    if (tE < (tI - 2.0)) {
        if (hE < 70.0) {
            corStatus = "#007bff";
            modoStr = "ECONOMIA ATIVA (VENTILACAO)";
        } else {
            corStatus = "#ffc107";
            modoStr = "SEGURANCA: UMIDADE ALTA";
        }
    }
}

String html = "<html><head><meta name='viewport'
content='width=device-width, initial-scale=1' charset='utf-8'>";
html += "<style>body{font-family:Arial; text-align:center;
background:#f0f2f5; margin:0;};";
html += "header{background:#198754; color:white; padding:20px;
font-size:24px; font-weight:bold;};";
html += ".container{padding:20px;} .card{background:white;
border-radius:15px; padding:25px; margin:10px; box-shadow:0 4px
10px rgba(0,0,0,0.1); display:inline-block; min-width:200px;};";
html += ".status{padding:20px; border-radius:10px; color:white;
font-size:1.2em; font-weight:bold; margin-top:20px; background:" +
corStatus + ";}</style>";
html += "<script>setInterval(function(){location.reload();},
3000);</script></head><body>";
html += "<header>IFCE - GESTOR TERMICO IOT</header>";
html += "<div class='container'>";
html += "<div class='card'><h3>Temp. Interna</h3><h2
style='font-size:3em; color:#333;'>" + String(tI, 1) + "°C</h2></div>";
html += "<div class='card'><h3>Temp. Externa</h3><h2
style='font-size:3em; color:#333;'>" + String(tE, 1) +
"°C</h2><p>Simulado via Potenciômetro</p></div>";
html += "<div><div class='status'>" + modoStr + "</div></div>";
html += "</div></body></html>";

server.send(200, "text/html", html);
```

```
}
```

```
// Funções de Suporte do LED RGB
```

```
void definirCor(int r, int g, int b) {
```

```
    analogWrite(PIN_R, r);
```

```
    analogWrite(PIN_G, g);
```

```
    analogWrite(PIN_B, b);
```

```
}
```

```
// Função de Gerenciamento do Wi-Fi (As 3 Opções de Conexão)
```

```
void conectar_wifi() {
```

```
    lcd.clear();
```

```
    lcd.print("1.Tentando Priv.");
```

```
    Serial.print("Tentando rede Privada...");
```

```
    WiFi.begin(wifi_privado, pass_privado);
```

```
    int tentativas = 0;
```

```
    while (WiFi.status() != WL_CONNECTED && tentativas < 10) {
```

```
        delay(500);
```

```
        Serial.print(".");
```

```
        tentativas++;
```

```
    }
```

```
// Se falhar a primeira, passa para a rede aberta
```

```
if (WiFi.status() != WL_CONNECTED) {
```

```
    lcd.clear();
```

```
    lcd.print("2.Tentando Aber.");
```

```
    Serial.print("\nTentando rede Aberta...");
```

```
    WiFi.begin(wifi_aberto, "");
```

```
    int tentativas_aberto = 0;
```

```
    while (WiFi.status() != WL_CONNECTED && tentativas_aberto < 10) {
```

```
        delay(500);
```

```
        Serial.print(".");
```

```
        tentativas_aberto++;
```

```
    }
```

```
}
```

```
// Se ambas falharem, cria o próprio Ponto de Acesso
```

```
if (WiFi.status() != WL_CONNECTED) {
```

```

        Serial.println("\nFalha nas redes. Ativando Access Point local...");
        WiFi.softAP(ap_nome, ap_pass);
        ip_atual = WiFi.softAPIP().toString();
        lcd.clear();
        lcd.print("3.Modos AP Ativo");
    } else {
        Serial.println("\nWiFi Conectado com sucesso!");
        ip_atual = WiFi.localIP().toString();
        lcd.clear();
        lcd.print("WiFi Conectado!");
    }
    delay(1500);
}

```

```

// =====
// SETUP E CONFIGURAÇÕES
// =====

```

```

void setup() {
    Serial.begin(115200);

    // Configuração dos Pinos de Saída
    pinMode(PIN_R, OUTPUT);
    pinMode(PIN_G, OUTPUT);
    pinMode(PIN_B, OUTPUT);
    pinMode(RELE_AC, OUTPUT);
    pinMode(RELE_MOTOR, OUTPUT);
    pinMode(POT_EXT, INPUT);

    // Inicia os relés desligados
    digitalWrite(RELE_AC, HIGH);
    digitalWrite(RELE_MOTOR, HIGH);

    lcd.init();
    lcd.backlight();

    // Executa o gerenciador de conexões
    conectar_wifi();

    server.on("/", handleRoot);
    server.begin();
}

```

```

    dhtInt.begin();
}

// =====
// LOOP PRINCIPAL
// =====
void loop() {
    server.handleClient();

    // Leitura segura do Sensor DHT
    float t_lida = dhtInt.readTemperature();
    if (!isnan(t_lida)) {
        tl = t_lida;
    }

    // Leitura analógica com média estável para o potenciômetro
    int somaPot = 0;
    for(int i = 0; i < 10; i++) {
        somaPot += analogRead(POT_EXT);
        delay(2);
    }
    int leituraPot = somaPot / 10;

    // Mapeia 0-4095 para 15.0°C - 45.0°C
    tE = map(leituraPot, 0, 4095, 150, 450) / 10.0;

    // --- LÓGICA DE EFICIÊNCIA ENERGÉTICA ---
    if (tE < (tl - 2.0)) {
        digitalWrite(RELE_AC, HIGH);    // DESLIGA Ar-Condicionado
        digitalWrite(RELE_MOTOR, LOW);  // LIGA o Motor/Ventilador
        definirCor(0, 0, 255);          // Azul
    } else {
        digitalWrite(RELE_AC, LOW);     // LIGA Ar-Condicionado
        digitalWrite(RELE_MOTOR, HIGH); // DESLIGA o
Motor/Ventilador
        definirCor(255, 0, 0);          // Vermelho
    }

    // --- ATUALIZAÇÃO DO DISPLAY LCD (IP EM CIMA, DADOS EM
BAIXO) ---

```

```

    lcd.setCursor(0, 0);
    lcd.print("IP:" + ip_atual);
    for (int i = ("IP:" + ip_atual).length(); i < 16; i++) {
        lcd.print(" ");
    }

    lcd.setCursor(0, 1);
    lcd.print("I:"); lcd.print(tI, 1); lcd.print("C E:"); lcd.print(tE, 1);
    lcd.print("C ");

    delay(20);
}

```

Versão final

```

#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "DHT.h"

// --- CREDENCIAIS DAS REDES ---
const char* wifi_privado = "Navegamais";
const char* pass_privado = "TonyS@les20";
const char* wifi_aberto = "IFCE-PECEM";

// Configuração do modo "Sem Internet" (Access Point)
const char* ap_nome = "PROJETO_EFICIENCIA";
const char* ap_pass = "12345678";

WebServer server(80);
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Definição de Pinos
#define PIN_R 4
#define PIN_G 2
#define PIN_B 15

```

```
#define RELE_AC 5      // Seu Relé do Ar-Condicionado
#define RELE_MOTOR 33  // Seu Relé do Motor / Ventilador
#define DHTPIN_INT 23
#define POT_EXT 34     // GPIO 34 para o Potenciômetro
#define DHTTYPE DHT22
```

```
DHT dhtInt(DHTPIN_INT, DHTTYPE);
```

```
float tl = 25.0, tE = 0, hE = 50.0;
String ip_atual = "";
```

```
// Variáveis para controlar a alternância do LCD sem travar o código
unsigned long tempo_anterior = 0;
bool mostrar_temperaturas = true;
```

```
// =====
// DASHBOARD WEB (HTML/CSS)
// =====
```

```
void handleRoot() {
    String corStatus = "#dc3545";
    String modoStr = "MODO PADRAO (AC LIBERADO)";
```

```
    if (tE < (tl - 2.0)) {
        if (hE < 70.0) {
            corStatus = "#007bff";
            modoStr = "ECONOMIA ATIVA (VENTILACAO)";
        } else {
            corStatus = "#ffc107";
            modoStr = "SEGURANCA: UMIDADE ALTA (AC
REALIMENTADO)";
        }
    }
}
```

```
String html = "<html><head><meta name='viewport'
content='width=device-width, initial-scale=1' charset='utf-8'>";
```

```

    html += "<style>body{font-family:Arial; text-align:center;
background:#f0f2f5; margin:0;}";
    html += "header{background:#198754; color:white; padding:20px;
font-size:24px; font-weight:bold;}";
    html += ".container{padding:20px;} .card{background:white;
border-radius:15px; padding:25px; margin:10px; box-shadow:0 4px
10px rgba(0,0,0,0.1); display:inline-block; min-width:200px;}";
    html += ".status{padding:20px; border-radius:10px; color:white;
font-size:1.2em; font-weight:bold; margin-top:20px; background:" +
corStatus + "};</style>";
    html += "<script>setInterval(function(){location.reload();},
3000);</script></head><body>";
    html += "<header>IFCE - GESTOR TERMICO IOT</header>";
    html += "<div class='container'>";
    html += "<div class='card'><h3>Temp. Interna</h3><h2
style='font-size:3em; color:#333;'>" + String(tI, 1) + "°C</h2></div>";
    html += "<div class='card'><h3>Temp. Externa</h3><h2
style='font-size:3em; color:#333;'>" + String(tE, 1) +
"°C</h2><p>Simulado via Potenciômetro</p></div>";
    html += "<div class='card'><h3>Umidade Interna</h3><h2
style='font-size:3em; color:#333;'>" + String(hE, 1) +
"%</h2></div>";
    html += "<div><div class='status'>" + modoStr + "</div></div>";
    html += "</div></body></html>";

```

```

    server.send(200, "text/html", html);
}

```

```

void definirCor(int r, int g, int b) {
    analogWrite(PIN_R, r);
    analogWrite(PIN_G, g);
    analogWrite(PIN_B, b);
}

```

```

void conectar_wifi() {

```

```

lcd.clear();
lcd.print("1.Tentando Priv.");

WiFi.begin(wifi_privado, pass_privado);
int tentativas = 0;
while (WiFi.status() != WL_CONNECTED && tentativas < 10) {
    delay(500);
    tentativas++;
}

if (WiFi.status() != WL_CONNECTED) {
    lcd.clear();
    lcd.print("2.Tentando Aber.");
    WiFi.begin(wifi_aberto, "");
    int tentativas_aberto = 0;
    while (WiFi.status() != WL_CONNECTED && tentativas_aberto <
10) {
        delay(500);
        tentativas_aberto++;
    }
}

if (WiFi.status() != WL_CONNECTED) {
    WiFi.softAP(ap_nome, ap_pass);
    ip_atual = WiFi.softAPIP().toString();
    lcd.clear();
    lcd.print("3.Modos AP Ativos");
} else {
    ip_atual = WiFi.localIP().toString();
    lcd.clear();
    lcd.print("WiFi Conectado!");
}
delay(1500);
}

```



```

// =====
// SETUP E CONFIGURAÇÕES
// =====
void setup() {
    Serial.begin(115200);

    pinMode(PIN_R, OUTPUT);
    pinMode(PIN_G, OUTPUT);
    pinMode(PIN_B, OUTPUT);
    pinMode(RELE_AC, OUTPUT);
    pinMode(RELE_MOTOR, OUTPUT);
    pinMode(POT_EXT, INPUT);

    digitalWrite(RELE_AC, HIGH);
    digitalWrite(RELE_MOTOR, HIGH);

    lcd.init();
    lcd.backlight();

    conectar_wifi();

    server.on("/", handleRoot);
    server.begin();
    dhtInt.begin();
}

// =====
// LOOP PRINCIPAL
// =====
void loop() {
    server.handleClient();

    // Leitura do Sensor DHT
    float t_lida = dhtInt.readTemperature();
    float h_lida = dhtInt.readHumidity();

```

```

if (!isnan(t_lida)) tl = t_lida;
if (!isnan(h_lida)) hE = h_lida;

// Leitura estável do potenciômetro
int somaPot = 0;
for(int i = 0; i < 10; i++) {
    somaPot += analogRead(POT_EXT);
    delay(2);
}
int leituraPot = somaPot / 10;
tE = map(leituraPot, 0, 4095, 150, 450) / 10.0;

// --- LÓGICA DE EFICIÊNCIA ENERGÉTICA ---
if (tE < (tl - 2.0)) {
    if (hE < 70.0) {
        digitalWrite(RELE_AC, HIGH);    // DESLIGA AC
        digitalWrite(RELE_MOTOR, LOW);  // LIGA o Ventilador
        definirCor(0, 0, 255);          // Azul
    }
    else {
        digitalWrite(RELE_AC, LOW);     // LIGA o AC
        digitalWrite(RELE_MOTOR, HIGH); // DESLIGA o Ventilador
        definirCor(255, 120, 0);         // Amarelo
    }
}
else {
    digitalWrite(RELE_AC, LOW);         // LIGA AC
    digitalWrite(RELE_MOTOR, HIGH);     // DESLIGA o Ventilador
    definirCor(255, 0, 0);               // Vermelho
}

// --- ATUALIZAÇÃO DO DISPLAY LCD ---
// Linha 1 fixa com o IP
lcd.setCursor(0, 0);

```

```

lcd.print("IP:" + ip_atual);
for (int i = ("IP:" + ip_atual).length(); i < 16; i++) {
    lcd.print(" ");
}

// Lógica de tempo para alternar a Linha 2 a cada 3000ms (3
segundos)
unsigned long tempo_atual = millis();
if (tempo_atual - tempo_anterior >= 3000) {
    tempo_anterior = tempo_atual;
    mostrar_temperaturas = !mostrar_temperaturas; // Inverte o
estado
    lcd.setCursor(0, 1);
    lcd.print("          "); // Limpa a linha antiga rapidamente antes
de reescrever
}

// Escreve na Linha 2 de acordo com o estado atual
lcd.setCursor(0, 1);
if (mostrar_temperaturas) {
    // Estado 1: Mostra as Temperaturas
    lcd.print("I:"); lcd.print(tI, 1); lcd.print("C E:"); lcd.print(tE, 1);
    lcd.print("C ");
} else {
    // Estado 2: Mostra a Umidade
    lcd.print("Umidade: "); lcd.print(hE, 1); lcd.print("% ");
}

delay(20);
}

```